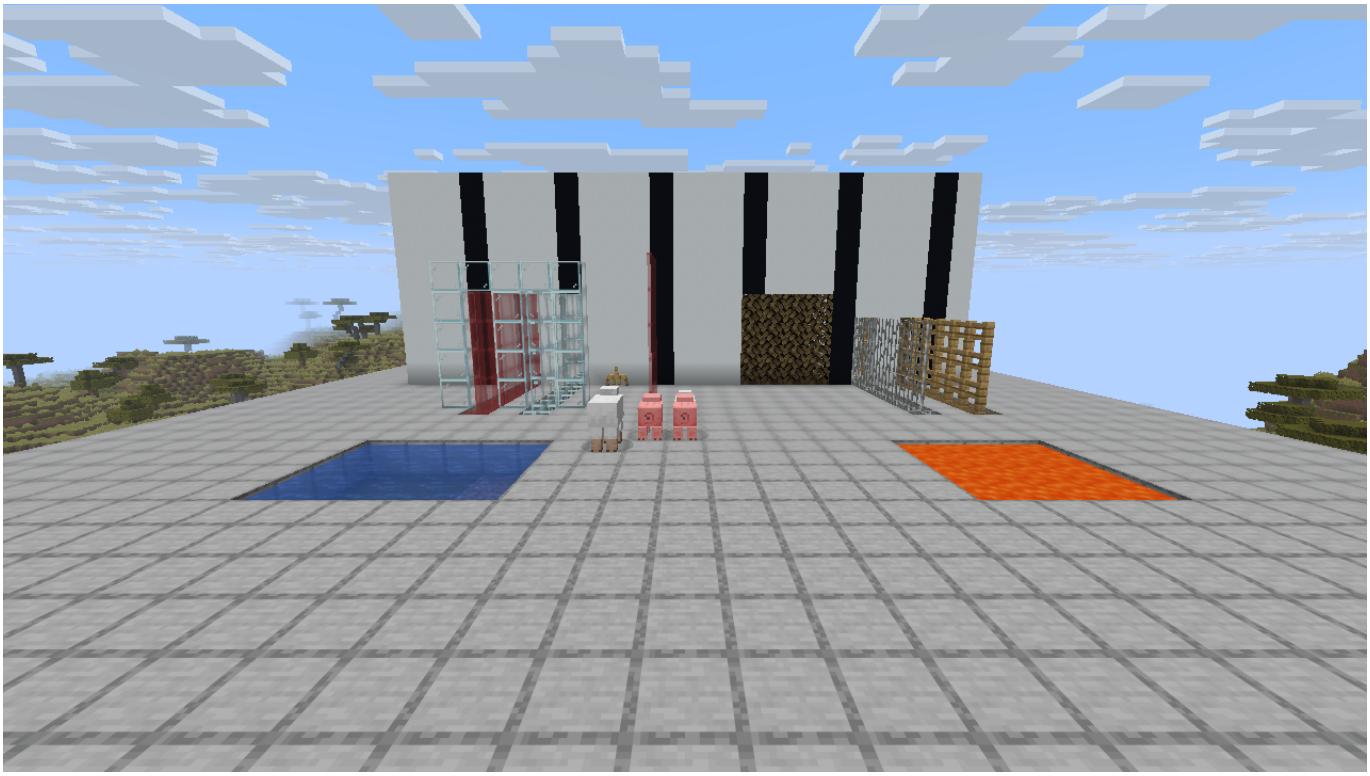
FABRIC 26.3 - CODE AND REAL-GAME AUDIT

Salt's Anti Aliasing

Image quality, temporal stability and implementation efficiency

Audit date: 24 October 2026 | Branch: fabric-26.3 | Commit: a3025cb



A working implementation with specific quality and lifecycle defects to address. The most reproducible visual issue is stationary TAA instability; the clearest integration gap is FSR1 without pre-antialiasing. Extreme SSAA targets also remain allocated after selecting Off.

What this report contains

Every exposed option is reviewed separately, including DLSS, FSR2, FSR3 and frame generation, plus the shared sharpening control. Evidence includes 1,664 fresh game screenshots, enlarged crops, GPU readbacks, a target-allocation inspection, 98 passing automated tests, and three randomized 1080p timing trials per mode.

This is an audit of the existing implementation. Production source files were preserved. Recommended changes have not been applied or validated as fixes.

READ THE EVIDENCE CORRECTLY

Scope, environment and method

Item	Audit configuration
Software	Minecraft 26.3; Fabric Loader 0.19.5; Fabric API 0.162.0+26.3; mod 1.1; Java 25.0.2 (Microsoft).
Hardware	RTX 3060 Ti, 8 GiB VRAM; NVIDIA driver 591.86; i7-12700KF; Windows 11; 32 GiB system RAM.
Rendering	Vulkan / RenderPearl. 1280 x 720 image comparisons; 1920 x 1080 repeated timing trials. VSync disabled, no inactivity throttle in measured samples.
Isolation	Separate copy of the existing test world in run/audit-26.3. Temporary capture/control harness in build/audit-20261024. Production code unchanged.
Native components	Existing staged production DLSS binaries and the bundled FidelityFX bridge/runtime were loaded. DLL hashes are recorded in the evidence manifest; native bridges were not rebuilt in this audit.

Test matrix actually executed

Off and all 11 exposed modes; fixed-camera static sequences; camera sweeps with scripted moving entities; MSAA 2x/4x/8x and alpha-to-coverage; SSAA 125%/200%/400%/800%; spatial Quality and Ultra Performance; temporal Quality, Performance and FSR Native AA; maximum sharpening; classic/improved transparency; night sky; an underwater glass tank; 720p-to-1080p resizing; and stationary precision near x=29,999,900.5.

Confidence labels

Observed means captured or measured in this run. Code-confirmed means the control/data flow is directly visible in the audited source. Risk / hypothesis means the concern still needs a targeted reproduction or an A/B experiment. A visual tradeoff is distinguished from an implementation defect.

Practical limits

One GPU/driver and a controlled scene cannot certify every world, material or mod combination. Sodium/Iris compatibility, first-person held items, high-refresh presentation latency, HDR and other vendors were not validated. FSR3 FG captures are of Minecraft's application render target: they do not expose the generated intermediate frames after the SDK swapchain. No artifact-free claim is made for those frames.

ISSUES, NOT JUST VISUAL PREFERENCES

Findings and priorities

ID	Priority / evidence	Finding
TAA-1	High / observed	Thin geometry crawls with the camera and world frozen. Repeated 32-frame crop variation is materially higher than other tested temporal modes.
FSR1-1	High / code + images	EASU receives an aliased scene; AMD specifies already-antialiased input. Thin rails remain coarse and unstable under motion.
TEMP-1	High / code + GPU	TAA/FSR/DLSS have camera-only reprojection. Moving-entity probes still read essentially zero motion. Color rejection cannot supply missing correspondence.
MEM-1	Medium / live inspection	An 800% SSAA scene target remains alive after switching Off: 450 MiB of color/depth payload at 720p output. Other inactive temporal targets are retained too.
SHARP-1	Medium / observed	Maximum generic sharpening produces bright/dark edge halos and harsh fine detail. It is a custom unsharp filter, not NVIDIA NVSharpen.
FSR-1	Medium / observed + code	Residual temporal instability and thin-detail loss remain, especially in lower-resolution presets. Mask quality and missing object motion are plausible contributors.
REC-1	Medium / code-confirmed	Several post-processing/DLSS failures leave sticky disabled flags while the saved selected mode can remain unchanged. No failure was induced in this audit.
COLOR-1	Medium / code risk	SSAA averages display-encoded RGBA8 values directly. A linear-light resolve may avoid dark mixed pixels; photometric A/B verification is still required.
PERF-1	Low / optimization candidates	Extra full-screen copies, broad barriers, scalar NIS work and unconditional SMAA Ultra warrant GPU profiling. The fastest replacement has not been demonstrated.
FG-1	Validation gap	Generated-frame distortion, HUD compositing and true output pacing require post-swapchain capture and presentation telemetry.

No fatal AA/native evaluation failure was logged during the successful audit session. One transient out-of-date swapchain warning occurred while resizing and recovered. Earlier setup failures were caused by disposable harness configuration and were corrected before measurements.

1080P - THREE RANDOMIZED TRIALS PER MODE

Measured frame intervals

The audit records wall-clock intervals between GameRenderer.render tail callbacks. These are application render-loop measurements, including scheduling and presentation backpressure. They are not isolated GPU pass durations, display latency, or a count of FSR-generated presents. Screenshot readback is disabled during timing samples.

Mode	Median ms	P95 ms	Trial median range
Off	0.625	1.030	0.619 - 0.629
FXAA	0.772	1.173	0.764 - 0.775
MSAA 4x	0.707	1.125	0.699 - 0.715
SSAA 200%	0.833	1.255	0.830 - 0.835
SMAA 1x Ultra	0.935	1.339	0.931 - 0.942
NIS Quality	0.824	1.227	0.816 - 0.833
FSR1 Quality	0.741	1.172	0.733 - 0.752
TAA	0.894	1.347	0.889 - 0.899
DLSS Quality	2.118	2.440	2.091 - 2.144
FSR2 Quality	1.738	2.066	1.731 - 1.749
FSR3 Quality	1.958	2.250	1.949 - 1.983
FSR3 + FG Quality	3.642	4.154	3.620 - 3.656

Each mode has 5,400 measured callback intervals, following 600 warm-up frames per trial. The order was shuffled with fixed seeds 263, 264 and 265. Scene, camera, output size and sharpening (0%) were held constant. Spatial and temporal upscalers used Quality; MSAA used 4x and SSAA used 200%.

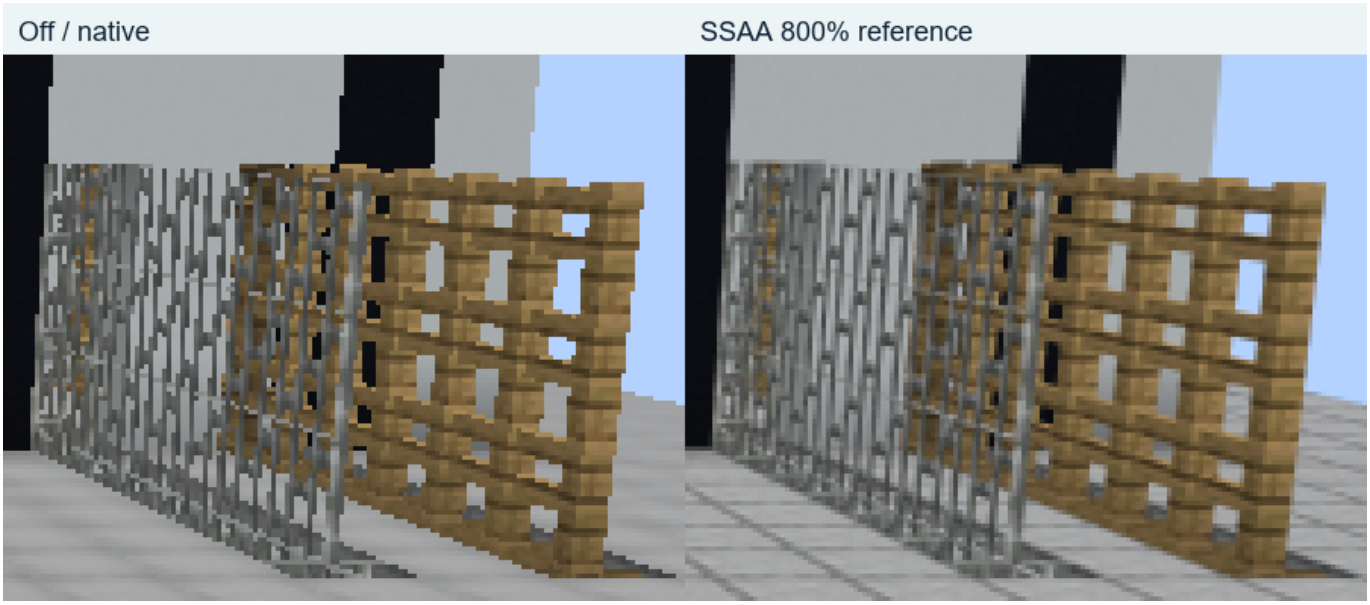
Interpretation

In this simple, frozen scene, native rendering is already inexpensive. FSR/DLSS and frame-generation bookkeeping add measurable overhead rather than guaranteeing a faster base render loop. This does not predict gains in a GPU-heavy world or shader pack. Profile per-pass GPU timestamps and present pacing before choosing optimizations; do not interpret these intervals as universal FPS rankings.

CONTROL CONDITION

Off / native baseline

Off is a comparison baseline, not an antialiasing technique.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The iron bars, fence silhouette, leaf cutouts and angled floor show native pixel stepping. In the frozen fence crop the 12-frame temporal standard deviation is 0.000 in 8-bit RGB code values. That confirms the control is stable; it does not mean native rendering is free of aliasing during motion.

Code assessment

Off skips the AA effects, but an independent positive sharpening value can still run the generic sharpening pass. The baseline tests explicitly set sharpening to zero. Mode switching can leave inactive render targets allocated; the Off label therefore does not imply minimum memory use (MEM-1).

Recommended work

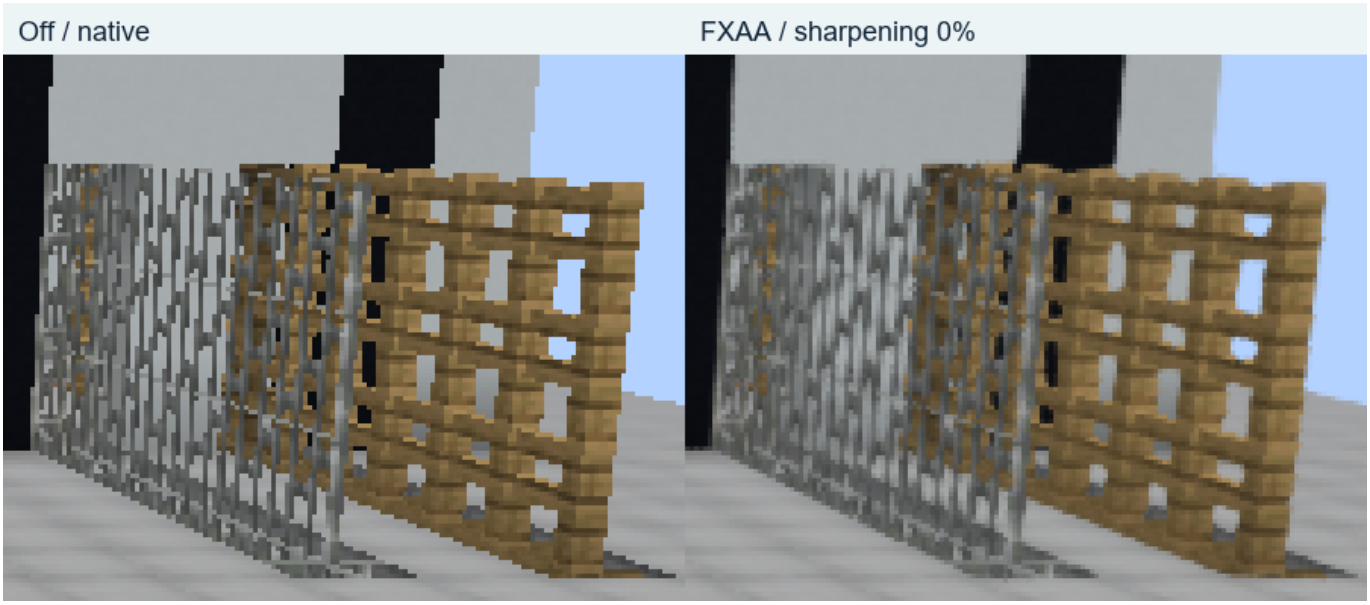
Keep a zero-sharpening Off preset for comparisons. Treat supersampled imagery as a useful geometric reference, while recognizing that LOD, filter shape and color-space handling can change texture appearance. Add an effective-render-path and allocation readout so the saved setting is distinguishable from actual runtime state.

Code: VulkanScenePostProcessor.java:85. Research: [S2], [S3]. Sources and links are in the reference section.

SPATIAL POST-PROCESS

FXAA

No specific reference-algorithm correctness fault was reproduced. The principal visual cost is detail softening.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

High-contrast fence and glass edges are smoothed, while thin bars and small texture features become softer. Spatial output is stable in the frozen sequence. Camera-motion captures still show the normal limits of a filter that cannot reconstruct subpixel geometry from prior frames.

Code assessment

The implementation uses the FXAA 3.11 Quality-style edge classification and bounded bidirectional search, with the preset-28 schedule. Bilinear sampling is enabled. Luma is derived from the display-referred scene; default SubpixelBlend is 0.75, with edge thresholds 0.125 and 0.0312. This placement is before HUD rendering.

Recommended work

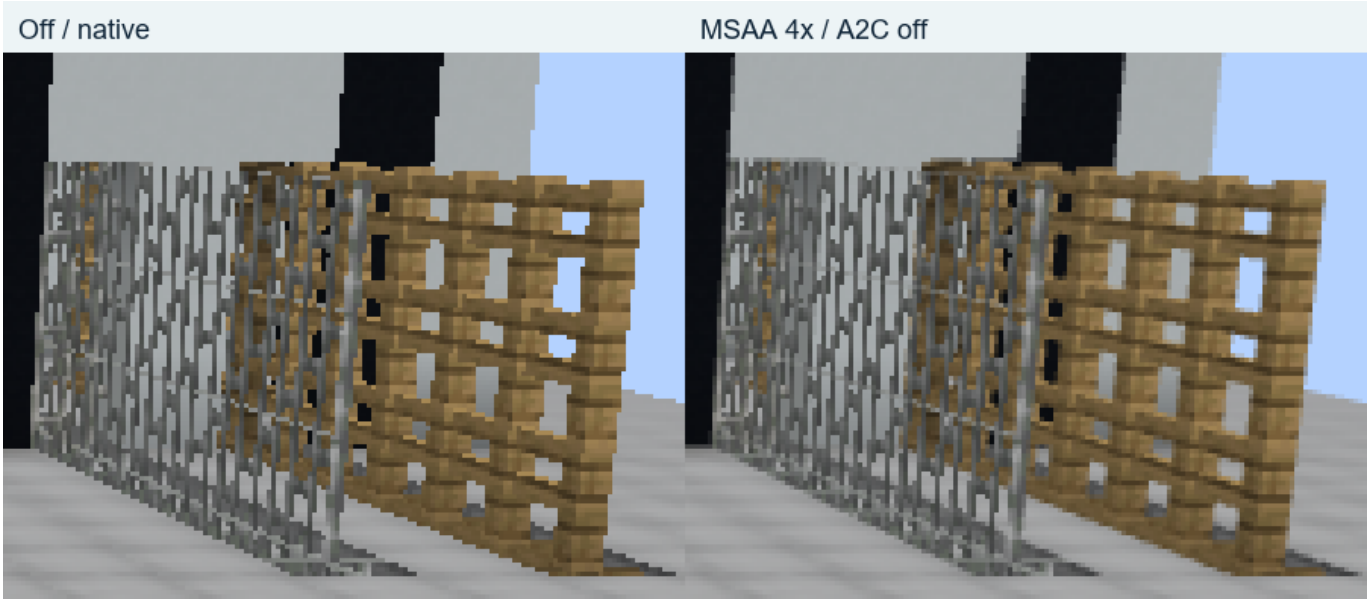
Offer a lower-subpixel-blend quality option and compare fine-texture retention against the current default; do not assume one value suits all Minecraft textures. Keep the early contrast exit. A final blit is an optimization candidate if a safe scene/HUD target handoff can eliminate it. Avoid compensating with 100% sharpening, which introduces halos (SHARP-1).

Code: fxaa.fsh:91; fxaa.json:17. Research: [S1]. Sources and links are in the reference section.

COVERAGE ANTIALIASING

MSAA

Geometry coverage resolves correctly in the sampled scenes; texture and cutout aliasing remain expected limitations.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

Fence silhouettes are smoother while interior texture detail is better preserved than with FXAA. The audit exercised 2x, 4x, 8x, optional alpha-to-coverage and improved transparency. No persistent broken depth/transparent ordering was found in these sampled images. Fine texture patterns are still aliased.

Code assessment

Stable compiled pipeline variants carry sample count into asynchronous native compilation. Reverse-Z depth resolves select MAX where supported, otherwise SAMPLE_ZERO, for single-sample transparency passes. Alpha-to-coverage is limited to non-blended cutout pipelines; the original alpha-discard defines remain. It is not a general fix for alpha-blended water/glass or shader aliasing.

Recommended work

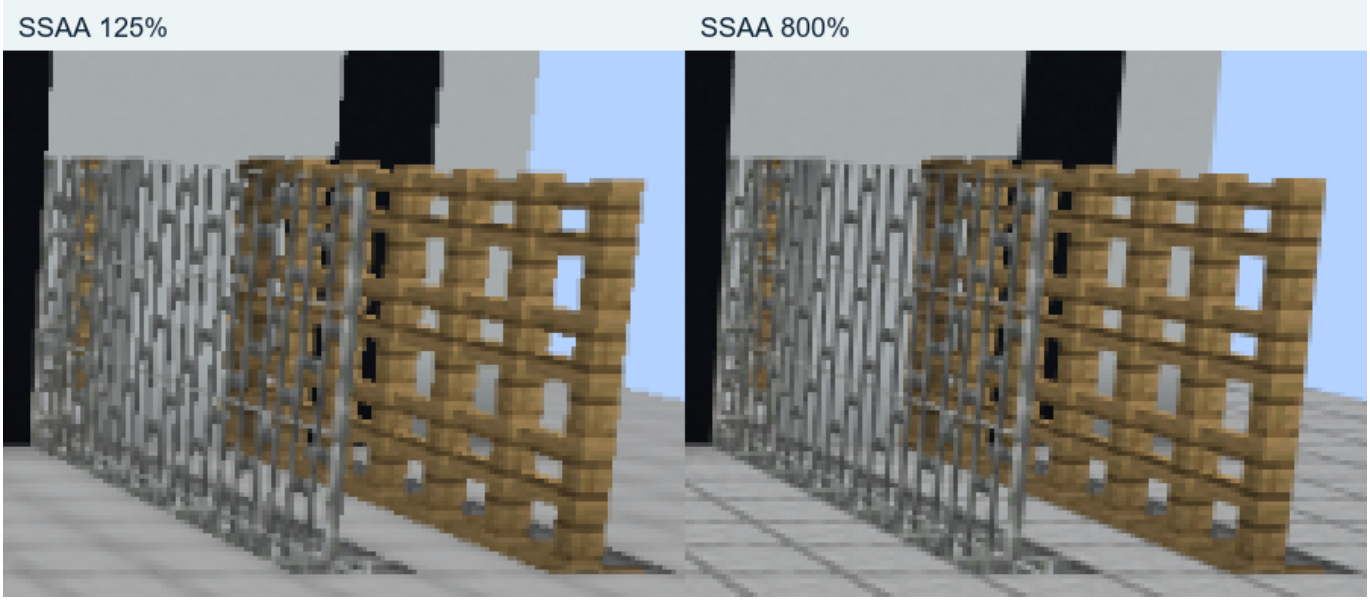
Keep 4x as the practical comparison point; use 8x only when measured image improvement justifies cost. Add a close-range and distant-leaf A/B suite for alpha-coverage preservation. Profile attachment load/store and separate color/depth resolves before changing them. Khronos tile-GPU bandwidth guidance motivates investigation, not a guaranteed desktop speedup.

Code: VulkanMsaaPipelineVariants.java:85; VulkanDepthResolver.java:53. Research: [S2], [S13]. Sources and links are in the reference section.

FULL-SCENE SUPERSAMPLING

SSAA

Area coverage is handled well, but high scales have a lifecycle defect and a color-space question.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

125%, 200%, 400% and 800% rendered successfully at 720p output. Higher scales smooth thin geometry. Fresh GPU checkerboard tests at 17-to-8, 32-to-8 and 64-to-8 pixels matched an area-weighted code-value reference with maximum errors of 0, 1 and 1 byte respectively. These tests validate covered-sample integration, not photometric correctness.

Code assessment

The resolve integrates the complete pixel footprint, including fractional coverage. However, RGBA8_UNORM scene values are averaged without sRGB decode/encode (COLOR-1). After 800% -> Off, live inspection found the 10240 x 5760 color/depth target retained: 450 MiB before allocation overhead (MEM-1). 800% means 64 times as many scene pixels, not eight times.

Recommended work

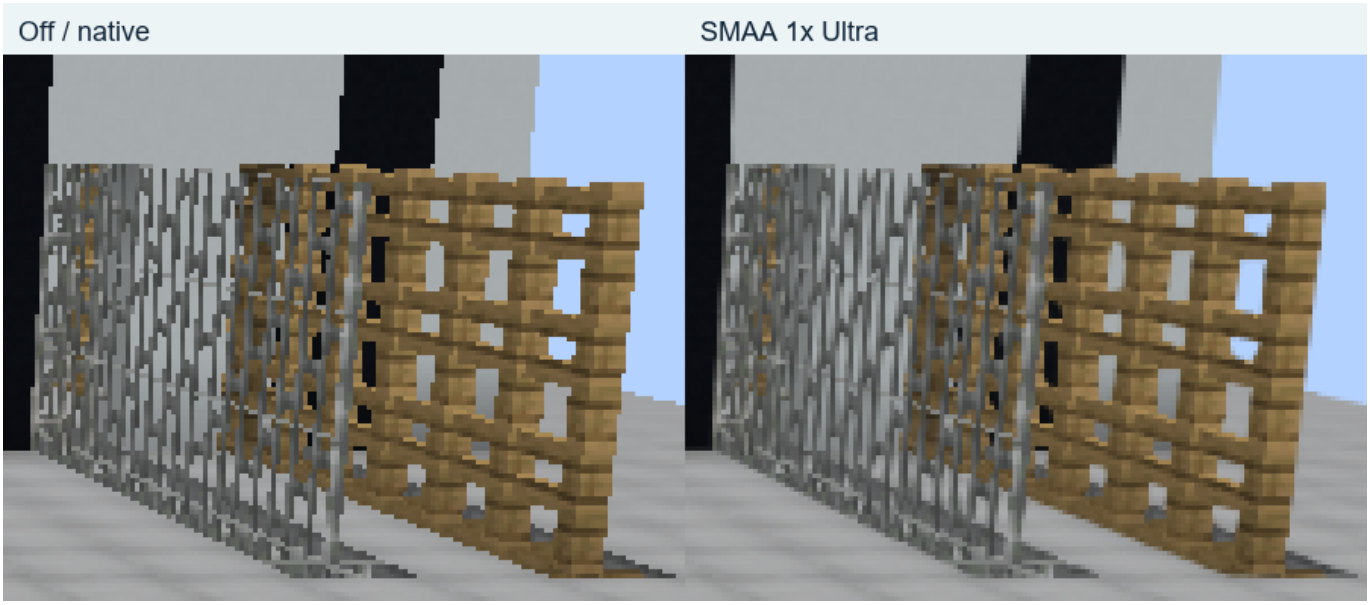
Release or budget large inactive targets after GPU work completes. Compare a linear-light area resolve against the current artistic/vanilla output using mixed black/white coverage. For high ratios, benchmark separable or hierarchical area reduction while retaining fractional-weight correctness. Wider reconstruction filters are an optional quality tradeoff; they can soften pixel-art textures.

Code: ssaa_resolve.fsh:9; VulkanSceneScaleController.java:71. Research: [S3], [S12]. Sources and links are in the reference section.

MORPHOLOGICAL ANTIALIASING

SMAA

The integration follows SMAA 1x Ultra. Remaining subpixel shimmer is not evidence of a broken temporal mode.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The image retains more local texture detail than FXAA while smoothing detected contours. The frozen sequence is stable. Moving-camera crops retain crawling on very thin features that are absent or only partially visible in a single source frame. No lookup-texture corruption or persistent full-screen artifact was observed.

Code assessment

The shipped path performs color edge detection, reference blend-weight calculation with horizontal/vertical/diagonal searches and corner handling, then neighborhood blending and a final blit. Lookup dimensions are 160 x 560 (area) and 64 x 16 (search). It is SMAA 1x, without temporal T2x accumulation. Ultra is selected directly in the shader.

Recommended work

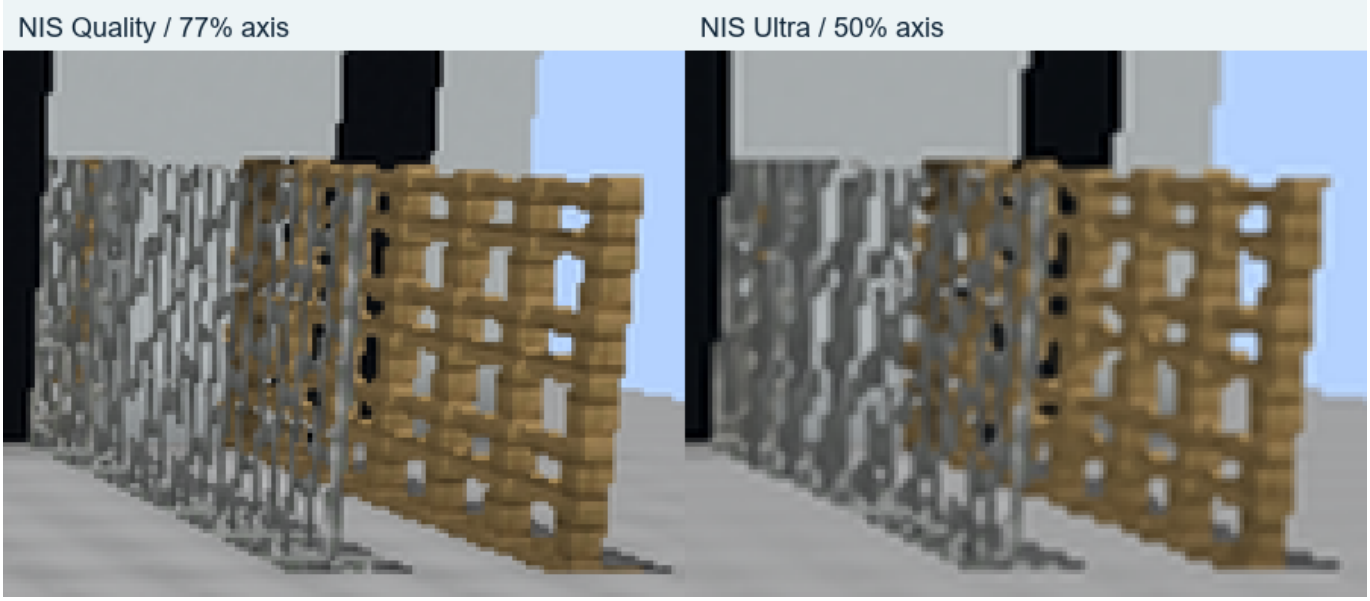
Expose High/Ultra choices if GPU profiling shows the longest searches matter. Consider edge/stencil work rejection and avoiding the final copy where the renderer permits it. A temporal SMAA variant would be a separate implementation requiring correct history and object motion; simply changing the label or reusing camera-only motion would not supply that quality.

Code: smaa_weight_blend.fsh:28; smaa.json:1. Research: [S4]. Sources and links are in the reference section.

SPATIAL RECONSTRUCTION

NIS Upscale

The scaler cannot replace missing AA. Low input resolution loses thin structure; scalar fragment work is an optimization candidate.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

At 50% per-axis input, rails and fence details become coarse or change thickness. The 77% Quality preset retains more structure. This is a spatial-only path: camera-motion aliasing is not stabilized through history. The observed low-resolution losses should be distinguished from the scale algorithm inventing geometry.

Code assessment

The adaptation uses the reference SDR coefficient bank and directional luminance filters, with bilinear chroma reconstruction. Each output pixel gathers a 6 x 6 luminance support into local arrays. It runs in a fragment pass rather than the SDK's tiled compute design. Reconstruction sharpening is removed, and the independent final filter owns sharpness.

Recommended work

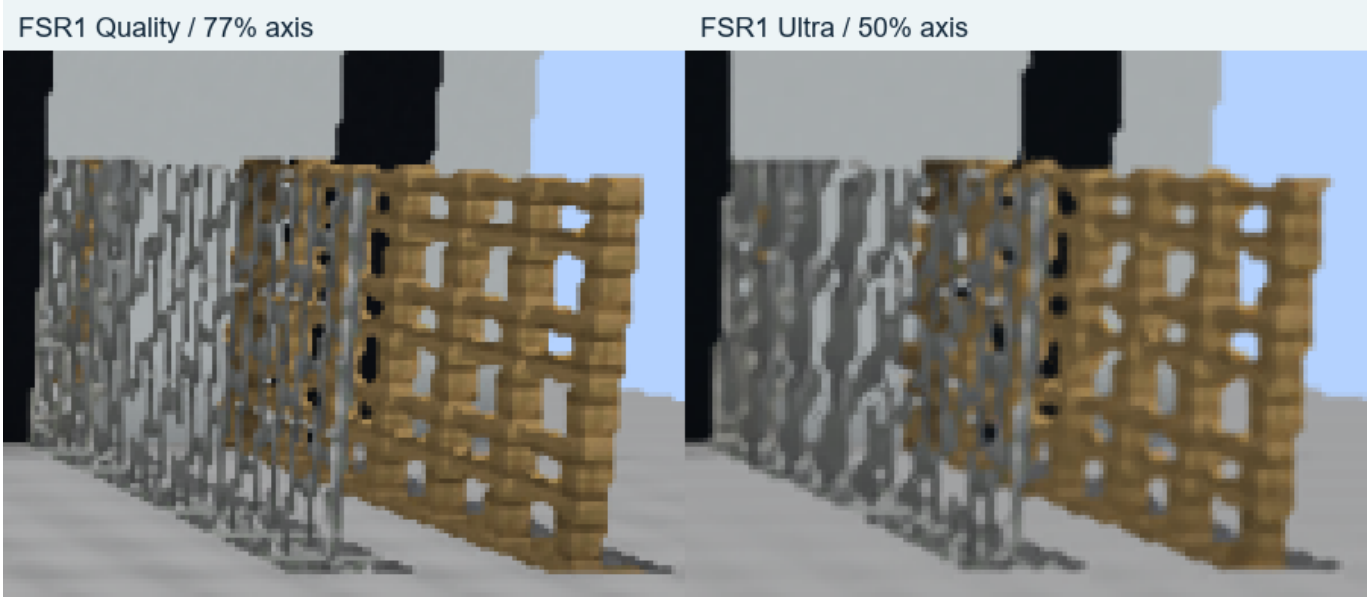
Offer an AA-before-spatial-upscale composition rather than presenting NIS alone as a complete AA solution. Investigate a compute implementation with shared neighborhoods/coefficient reuse and appropriate precision, but only after GPU timing and register-pressure analysis. Preserve the current SDR color-space contract. If using the full NVScaler later, avoid accidentally adding a second sharpen stage.

Code: nis_upscale.fsh:10; nis_scaler.glsl:1. Research: [S5]. Sources and links are in the reference section.

SPATIAL RECONSTRUCTION

FSR1 Upscale

FSR1-1: the EASU implementation is given aliased input, contrary to AMD's integration guidance.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

Thin bars and fences become coarse at the lower input scale. In the motion sequence the spatial path does not stabilize the detail. Quality retains more structure than Ultra Performance. These images show the practical effect, but do not establish how much a particular pre-AA technique will recover until that change is tested.

Code assessment

The code runs the FP32 reference-style EASU after reduced-resolution world rendering. The final-effects switch applies FXAA/SMAA only when those modes are selected, so FSR1 has no preceding AA stage. AMD explicitly calls for already-antialiased, perceptual-space input. No upscale-specific MIP bias control was found in the audited mod source.

Recommended work

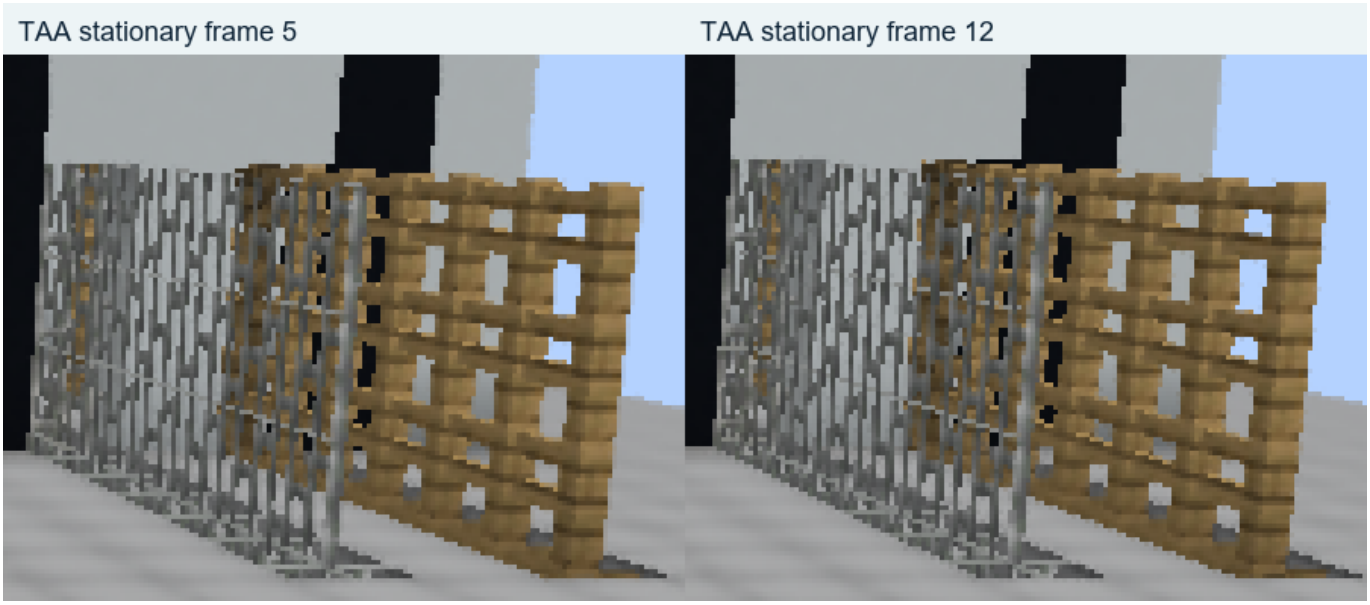
Add a composable pre-AA stage (for example, internal-resolution SMAA/MSAA or a validated TAA path), then EASU, then optional sharpening. Tune MIP bias alongside shimmer rather than blindly increasing sharpness. Keep the perceptual [0,1] input contract. The current generic sharpening control should not be described as AMD RCAS.

Code: VulkanSceneScaleController.java:379; VulkanScenePostProcessor.java:192. Research: [S6]. Sources and links are in the reference section.

TEMPORAL ANTIALIASING

TAA

TAA-1: stationary thin-detail instability is a reproducible image-quality defect.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

With the world frozen and camera fixed, a 32-frame repeat produced mean per-channel temporal standard deviation of 7.23/255 in the fence crop, with a maximum channel standard deviation of 108.18. The initial 12-frame run independently showed 5.29/255; non-temporal controls were exactly stable. Enlarged images show thin edge/detail changes, not just an average-image blur.

Code assessment

The shader uses 16-phase Halton jitter, camera reprojection, reverse-Z depth checking, YCoCg variance clipping, Catmull-Rom history filtering and a 0.90 base history weight. Luma/clipping confidence can sharply reject history at high-contrast jittered edges. That is a plausible contributor, not an experimentally isolated root cause. There are no per-entity motion vectors.

Recommended work

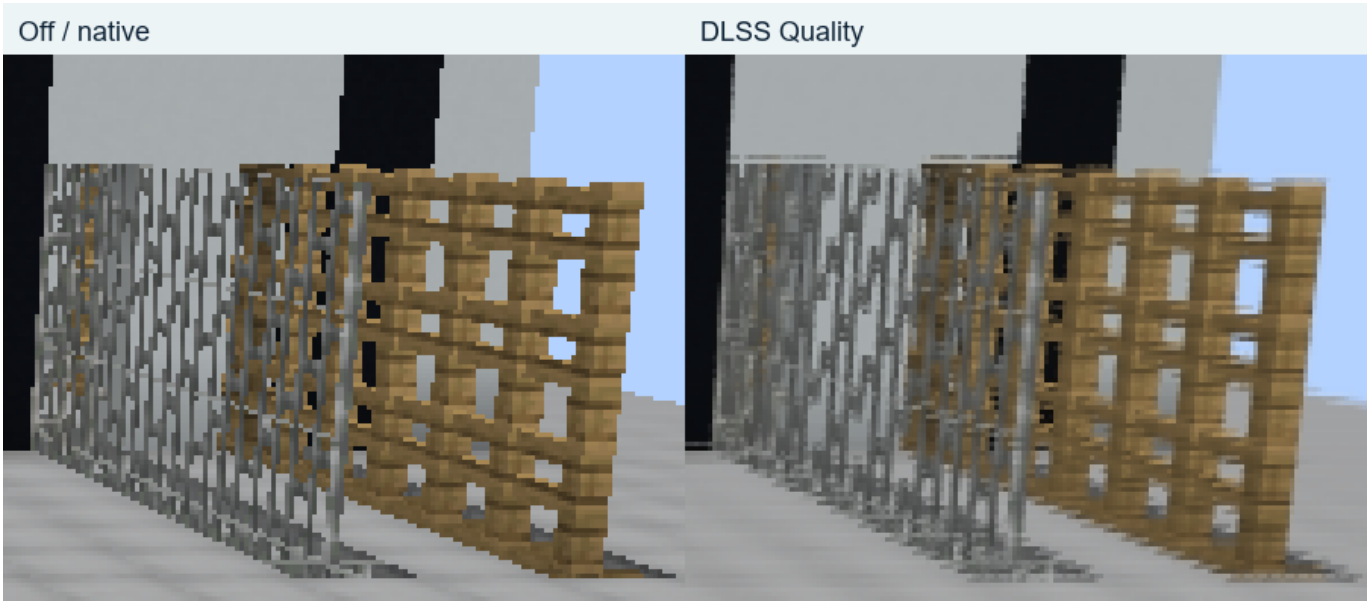
Add debug views for effective history weight and each rejection term. Verify current/history sample alignment and jitter sign with a static synthetic diagonal before tuning. Test less aggressive rejection and a filtered current reconstruction against edge stability, disocclusion and ghosting. Add object motion for moving geometry. A successful fix must improve the stationary test without extending trails in motion.

Code: taa_resolve.fsh:177; VulkanSceneTemporalController.java:109. Research: [S7]. Sources and links are in the reference section.

VENDOR TEMPORAL RECONSTRUCTION

DLSS Super Resolution

Stationary quality is strong in this scene. Camera-only motion and failure recovery remain implementation concerns.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The repeated 32-frame fixed-camera crop varied by 0.386/255 on average. Quality and Performance, motion, glass/water, improved transparency, underwater views, night sky and world-border coordinates ran. No persistent severe trails were identified in the sampled moving-entity contact sheets, but this is not proof of complete motion correctness.

Code assessment

The controller queries SDK optimal sizes; the bridge supplies pixel-space jitter, normalized pixel-motion scaling, reverse-Z constants and an R16F current-color-bias hint. Its motion is reconstructed from scene depth and camera matrices. With scripted entity motion and a fixed camera, GPU motion stayed below 0.000015 pixels: no entity correspondence is provided (TEMP-1). Evaluation/setup exceptions can set a persistent disabled flag (REC-1).

Recommended work

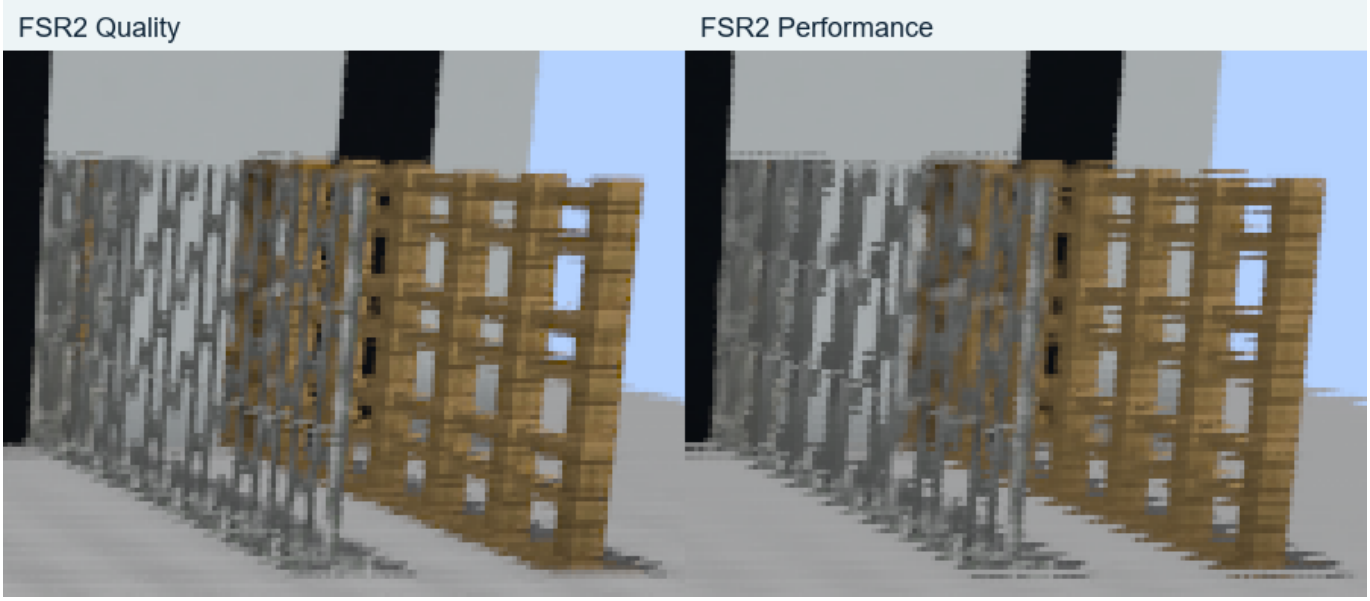
Provide previous/current object transforms or a dedicated dynamic velocity pass; retain the color hint as a supplement. Test thin moving objects against similar-color backgrounds, where color rejection can miss changes. Add explicit effective status and controlled recovery on renderer/configuration changes. Profile broad barriers and copies before combining native work into fewer submissions.

Code: temporal_motion.glsl:1; VulkanSceneDlssController.java:104. Research: [S8], [S9]. Sources and links are in the reference section.

TEMPORAL RECONSTRUCTION

FSR2 Super Resolution

Residual instability and missing dynamic correspondence warrant further quality work.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The initial frozen Quality fence sequence varied by 0.805/255; the 32-frame Native AA repeat varied by 2.20/255. These are different presets and sample counts, not a fair single-number quality ranking. Some thin detail changes between frames, and Performance loses more fine structure. No fatal native evaluation error was logged.

Code assessment

The mod decodes scene/opaque color to linear FP16, builds reactive and composition masks, provides unjittered camera motion and invokes the native SDK. Full and opaque color histories help infer changes, but they do not replace velocities for moving opaque, alpha-tested and animated geometry. AMD recommends broad object/material motion coverage and explicit reactivity for transparency.

Recommended work

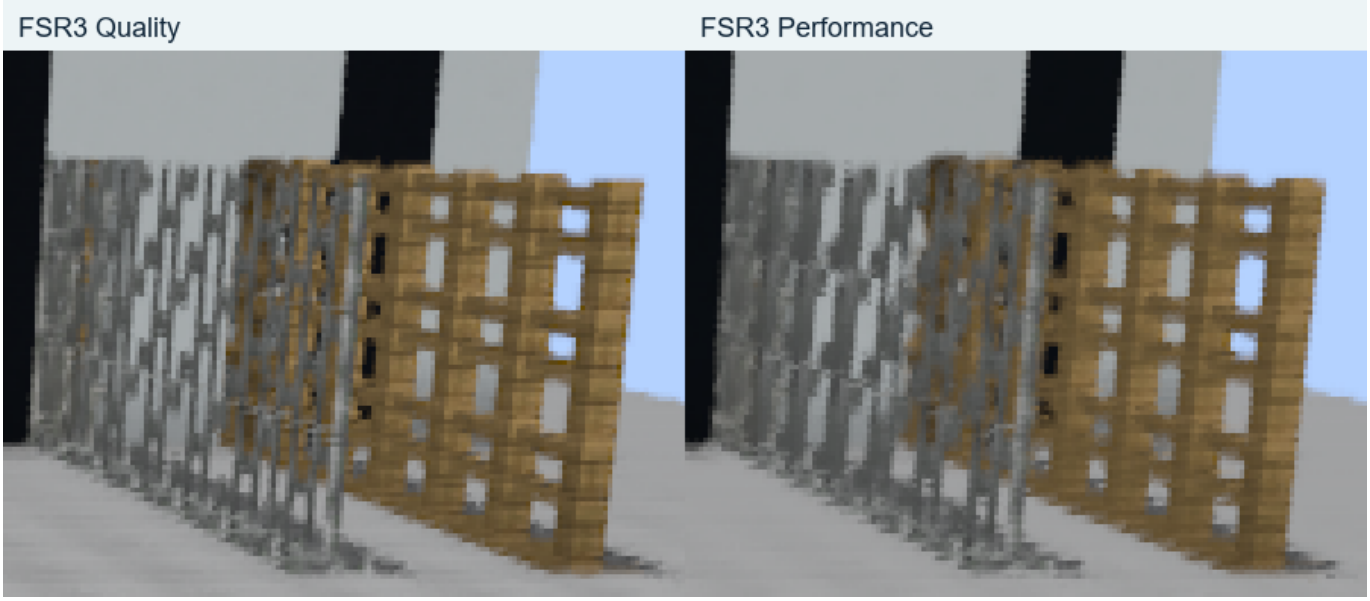
Prioritize object motion and inspect masks over glass, particles and animated textures. Validate jitter/depth alignment and masks at Native AA separately from upscaling. Test quality-aware MIP bias with temporal shimmer controls. Profile the two decodes, history copies, motion/mask generation, native dispatch and output conversion; use per-pass GPU evidence before attempting fusion.

Code: fsr_temporal_masks.fsh:83; VulkanSceneFsrController.java:814. Research: [S9]. Sources and links are in the reference section.

TEMPORAL RECONSTRUCTION

FSR3 Super Resolution

The FSR3 path is distinct from frame generation; it shares the camera-only motion limitation.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The 12-frame Quality crop varied by 0.543/255. The Native AA 32-frame repeat varied by 0.440/255. Lower-resolution Performance images lose thin rail/fence detail. Sampled underwater and improved-transparency paths completed without a persistent scene-wide artifact.

Code assessment

The native bridge chooses an SDK version ID for FSR3 while sharing the Java scene preparation with FSR2. History resets account for configuration, timing discontinuities and frame sequence changes. Sharpening is delegated to the SDK when evaluation succeeds, avoiding a redundant generic sharpen. Dynamic object velocities remain absent; inferred color masks are an approximation.

Recommended work

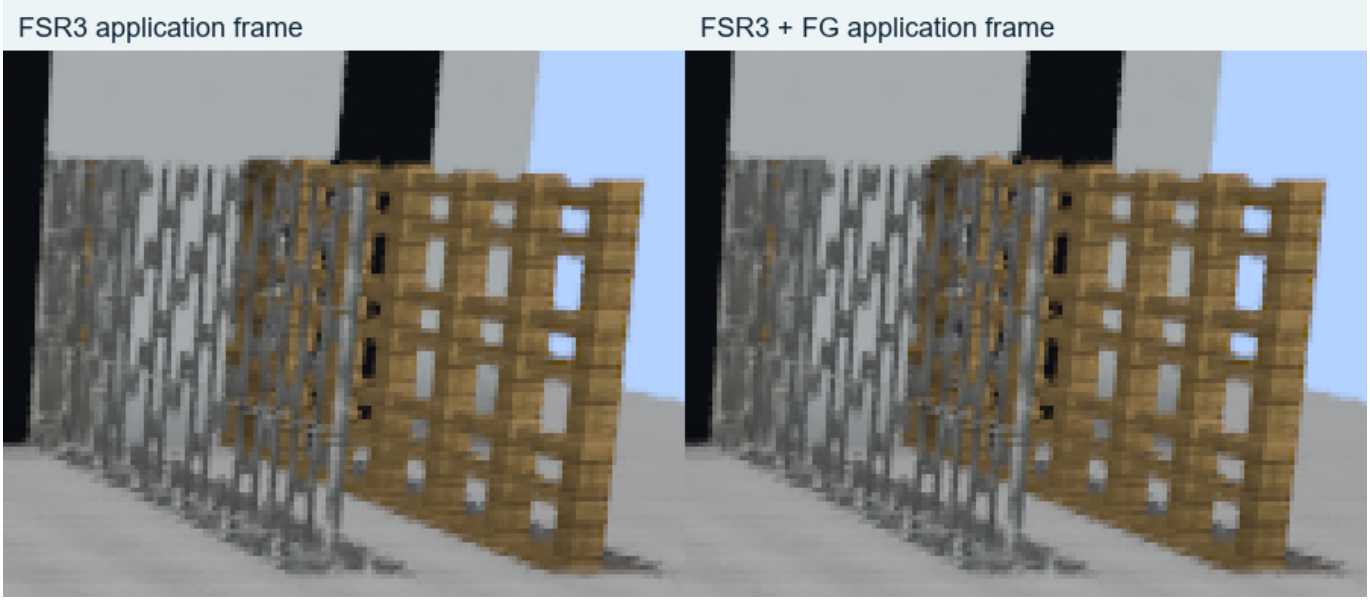
Use the same object-motion and material-mask improvements as FSR2, then validate this SDK version independently rather than assuming parity. Keep Native AA available for high image-quality use cases. Profile conversion/copy overhead and FP16 buffer lifetimes. Add automated camera cuts, close disocclusions and animated emissive textures to the real-game regression suite.

Code: VulkanSceneFsrController.java:539; salts_fsr_bridge.cpp:1495. Research: [S9], [S10]. Sources and links are in the reference section.

PRESENTATION AND INTERPOLATION

FSR3 + Frame Generation

Base rendering ran successfully; generated-frame artifact freedom was not verified by render-target screenshots.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

The FG mode rendered during static, moving-camera/entity, improved-transparency, underwater and resize tests. Captured application frames remained coherent. The additional application-loop overhead appears in the timing table. Those measurements neither count interpolated presents nor demonstrate reduced display latency.

Code assessment

The integration uses the SDK replacement swapchain, queue roles, frame preparation and a HUD-less scene copy. HUD-less automatic separation is a supported integration route, but AMD documents potential artifacts around semitransparent UI. The screenshots in this report are taken before the proxy swapchain produces intermediate frames, and HUD was hidden for image comparisons.

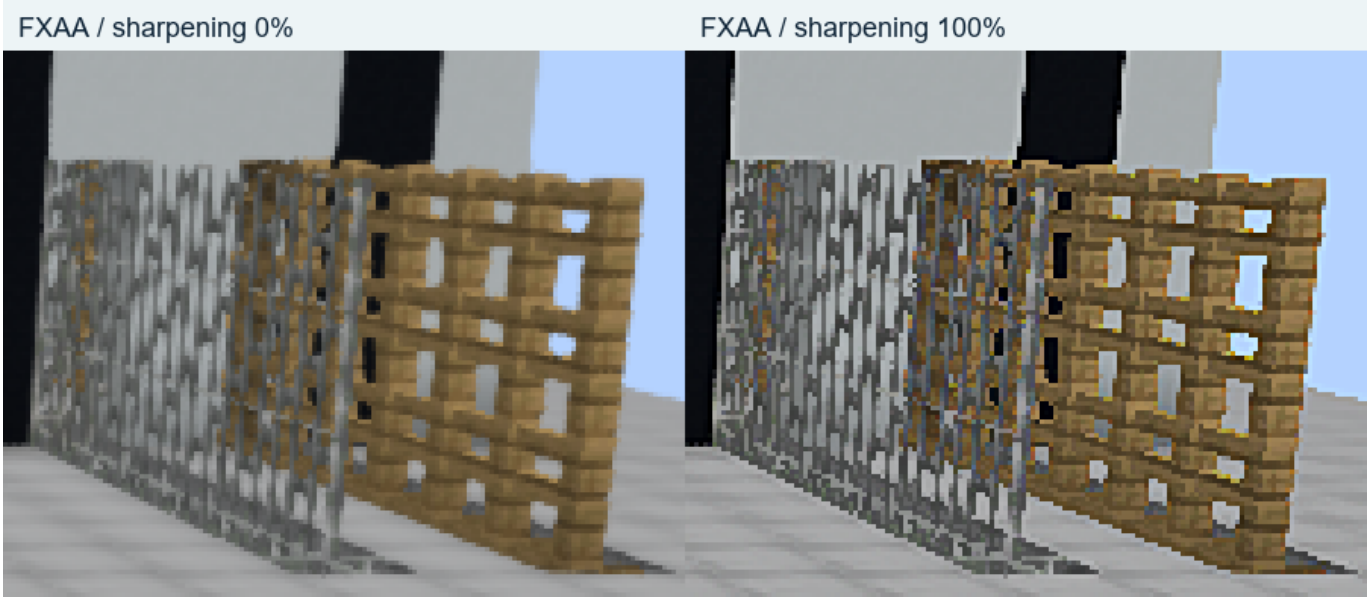
Recommended work

Before declaring FG release-ready, capture the actual presented output with frame IDs at sufficient temporal resolution. Test moving thin geometry, disocclusion, water, text, crosshair, chat and translucent menus. Add rendered/generated present counts, pacing and latency telemetry. Consider separate UI composition or a presentation callback. AMD recommends at least 60 base fps before interpolation; generated frames cannot repair missing object motion.

Code: salts_fsr_bridge.cpp:1516; RenderRuntime.java:342. Research: [S10], [S11]. Sources and links are in the reference section.

Independent sharpening

SHARP-1: the 100% setting creates visible halos. The general filter is custom NIS-style sharpening.



Unretouched in-game crops, source rectangle (790, 270) to (939, 389) at 1280 x 720. Enlarged 3x with nearest-neighbor sampling.

Observed in the game

Maximum sharpening produces bright and dark outlines around fence rails, increases harsh texture contrast and accentuates stair steps. The same stress setting on SMAA produces similar overenhancement. Default zero sharpening avoids this artifact. Sharpening is not a substitute for additional image samples.

Code assessment

The generic shader is a 3 x 3 Gaussian-style unsharp mask with gain proportional to Sharpness x 4, an edge boost and neighborhood padding of 0.18. It is not the SDK's directional NVSharpen or AMD RCAS. FSR2/3 use native sharpening after successful evaluation. Legacy NIS Sharpen, SMAA + NIS Sharpen and FSR1 + RCAS serialized modes migrate to a base mode plus the shared control.

Recommended work

Replace the generic filter with a validated directional/adaptive sharpener, or name it as a custom filter. Reduce overshoot allowance and test a conservative strength curve. Keep the zero default and avoid double sharpening if adopting a complete vendor scaler. Validate bright/dark steps, thin bars, low-contrast surfaces and temporal stability at multiple strengths, not only at 100%.

Code: nis_sharpen.fsh:77; VulkanDynamicUniforms.java:26. Research: [S5]. Sources and links are in the reference section.

Resource lifetime and runtime truth

MEM-1 - large inactive targets remain reachable

A temporary JVM inspection agent read the actual controller fields on Minecraft's execution thread after SSAA 800% -> Off. It did not change production rendering code. The scene target remained 10240 x 5760, RGBA8_UNORM plus D32_FLOAT, sample count 1. Pixel payload is 10240 x 5760 x (4 + 4) = 471,859,200 bytes = 450 MiB. This is an allocation estimate from live dimensions, not a measured process-VRAM total.

Off-state retained targets	Evidence / consequence
SSAA scene	10240 x 5760 color + depth after selecting Off. Large allocations can remain until another scaled target replaces them.
TAA history	1280 x 720 RGBA8 color + depth retained after TAA is inactive.
DLSS targets	Scene, output, motion, previous color and rejection resources remained referenced.
FSR targets	Scene/opaque/linear histories, motion, masks, upscale output and HUD-less target remained referenced.
MSAA	No retained MSAA scene target appeared in this Off-state snapshot; this finding is not attributed to MSAA.

Recommendation: explicit deactivation hooks, deferred destruction after GPU completion, and a small bounded reuse budget. Do not destroy in-flight textures synchronously. Inspect high-resolution OIT/FSR attachments too; the captured inventory is not a complete driver memory accounting.

REC-1 - selected setting can outlive the effective renderer

DLSS and post-processing controllers can set sticky disabled-after-failure flags. The saved mode is returned by RenderRuntime.activeMode; that alone does not certify a pass executed. FSR already has a safer configuration-specific recovery mechanism that can inform a consistent approach. This audit encountered no native evaluation failure, so recovery behavior is a source finding rather than a reproduced crash.

PERF-1 - profile before rewriting

The NIS fragment path repeats neighborhood work; several post chains add a copy-back pass; temporal native paths have multiple full-screen conversions/copies and broad barriers. These are candidates for GPU timestamp and bandwidth profiling. Also review the strong static LAST_UPLOADED_HASHES map when shader buffers are recreated; stale entries should be evicted with resource lifetime. No speedup or heap-leak rate was measured for a replacement.

Code: VulkanSceneScaleController.java:71; VulkanSceneTemporalController.java:272; VulkanSceneDlssController.java:57; VulkanDynamicUniforms.java:32

Temporal evidence in detail

Sequence / preset	Frames	Mean channel SD	Max channel SD
TAA / native	32	7.228	108.183
DLSS / Quality	32	0.386	18.246
FSR2 / Native AA	32	2.196	72.298
FSR3 / Native AA	32	0.440	46.652

Units are 8-bit RGB code values, 0-255. Each pixel/channel's standard deviation is computed over the actual saved frames, then averaged over the 150 x 120 fence crop. Camera position/rotation and world simulation were fixed. This measures residual image variation, not perceived overall quality. Presets differ and a completely blurred image could score artificially well; assess the enlarged images alongside these values.

TEMP-1 - motion probes expose the missing signal

While tagged entities moved in front of a fixed camera, the maximum value anywhere in the DLSS, FSR2 and FSR3 GPU motion textures stayed under 0.000015 pixels. That agrees with the shader's camera/depth-only model. Opaque object motion is not encoded. Color-change rejection can reduce stale history but cannot track a moving surface with similar foreground/background color or preserve its subpixel detail.

Moving entity / DLSS, frame 10



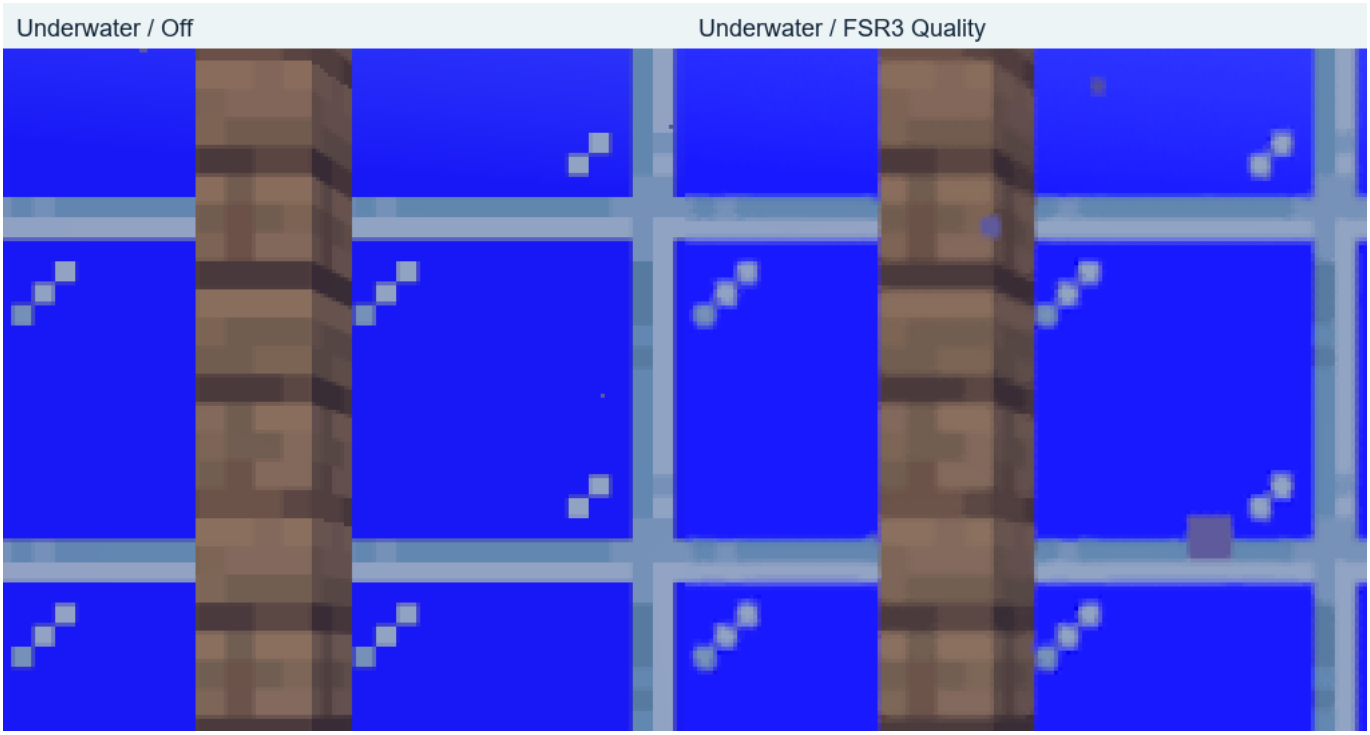
Moving entity / DLSS, frame 30



Actual sequence frames at a fixed camera. The harness moves tagged entities along a scripted path using server teleports; these are stress tests, not a benchmark of normal walking animation.

A new velocity pass should include previous/current object transforms and relevant animation/alpha-tested motion, with camera motion handled exactly once. Preserve reverse-Z and unjittered vector conventions. Validate with per-pixel debug overlays, not just a global mean. Research: [S7], [S8], [S9]. Sources and links are in the reference section.

Additional game checks



Underwater glass-tank detail, cropped from fresh 1280 x 720 captures. Improved transparency enabled; nearest-neighbor enlargement only.

The underwater scene includes water, glass, a fence and iron bars. Every exposed mode produced captured output. Night-sky sequences covered Off, MSAA, SSAA, TAA, DLSS, FSR2 and FSR3, including clear-depth sky and clouds. These samples showed no persistent full-screen corruption. Small stars and thin edges still need temporal sampling; a finite screenshot set cannot exclude all flicker.

World-border precision

Mode	Max stationary motion (px)	Non-finite pixels
DLSS	0.00001451	0
FSR2	0.00001451	0
FSR3	0.00001454	0

These GPU readbacks were captured near x=29,999,900.5 after camera settling. The camera-relative matrix implementation avoids the large absolute-coordinate precision loss seen in naive float world matrices. Passing this check does not test moving entities or every camera path.

Resizing and synthetic resolve checks

The active game resized between 1280 x 720 and 1920 x 1080 and returned to 720p. An out-of-date swapchain warning recovered during the transition. Fresh in-game checkerboard readbacks verified fractional SSAA footprint coverage to one 8-bit level. Neither test validates every driver or every generated presentation frame.

CONCRETE ACCEPTANCE CRITERIA

Recommended implementation sequence

Order	Work	Acceptance test
1	Repair stationary TAA stability	Instrument each rejection term; freeze camera/world and run at least a full jitter cycle. Improve the thin-edge variation without simply blurring the image or adding motion trails.
2	Add dynamic velocities	A fixed camera with moving geometry produces correct nonzero per-object motion. Verify sign, scale, jitter exclusion, deformation and disocclusion.
3	Compose AA before FSR1/NIS	Compare internal-resolution AA + spatial upscale against current input on rails, foliage and transparent surfaces. Measure extra pass cost.
4	Bound inactive GPU resources	800% SSAA -> Off releases/evicts the large scene target after GPU completion. Repeat mode/size changes and verify a stable allocation budget.
5	Improve sharpening	Use a documented adaptive kernel or a clearly named custom filter. Limit halos on step edges and compare strengths at equal perceived sharpness.
6	Make runtime state observable	Expose requested/effective mode, fallback reason and target sizes. Reproduce a shader/native failure and recover safely on an explicit retry or valid configuration change.
7	Profile GPU work	Collect timestamps, bandwidth, register use and present intervals. Optimize the demonstrated bottleneck; revalidate screenshots after every fusion/precision change.
8	Complete FG validation	Record post-swapchain real/generated frames with IDs, UI visible, and sufficient capture rate. Check pacing/latency at low and high base frame rates.

Suggested coverage extensions: AMD/Intel devices; current and alternate NVIDIA drivers; Sodium and other renderer combinations; 1440p/4K; first-person hands and held items; moving translucent particles; emissive/animated textures; resource reload recovery; GUI scale changes and semitransparent menus. These are outstanding coverage items, not claims of defects.

READ ON 24 OCTOBER 2026

Primary research sources

Implementation recommendations were checked against original vendor/author references. The report's measurements and code-specific findings are original audit evidence. Documentation versions may differ from the bundled native runtime; match the runtime headers before applying SDK-specific changes.

[S1] NVIDIA FXAA whitepaper

Perceptual edge filtering and quality/performance tradeoffs.

[S2] Khronos Vulkan Samples: MSAA performance

Coverage versus shading, attachment resolves and bandwidth. Tile-GPU figures are not transferred to this desktop GPU.

[S3] GPU Gems 2, Chapter 21: High-Quality Antialiased Rasterization

Supersampling, reconstruction filters and separable downsampling.

[S4] SMAA authors: project and integration reference

Original three-stage algorithm, lookup textures and quality presets.

[S5] NVIDIA Image Scaling: NIS_Scaler.h, SDK 1.0.3

Directional scaler/sharpener, input color space, thread groups and precision options.

[S6] AMD GPUOpen: FidelityFX Super Resolution 1.2

AA input, perceptual colors and texture MIP bias requirements.

[S7] Yang, Liu and Salvi: A Survey of Temporal Antialiasing Techniques (2020)

Temporal sampling, history validation and reconstruction tradeoffs.

[S8] NVIDIA Streamline: DLSS integration guide

Optimal settings, pixel-space jitter, motion scaling, constants and resource tagging.

[S9] AMD GPUOpen: FidelityFX Super Resolution 2.3.3

Object motion vectors, reactive/composition masks and depth conventions.

[S10] AMD GPUOpen: FSR 3.1.4 upscaling and frame generation

Frame preparation, presentation, UI alternatives and synchronization.

[S11] AMD GPUOpen: FSR 3 product/developer guidance

Recommended pre-interpolation base frame rate of at least 60 fps.

[S12] GPU Gems 3, Chapter 24: The Importance of Being Linear

Why averaging display-encoded values can darken mixed pixels.

[S13] Khronos Vulkan specification: rasterization

Sample count and alpha-to-coverage behavior.

AUDIT HANDOFF

Reproduction and evidence inventory

Audited commit: a3025cbba534d83a7c9ae3d7708c8e0c191b516c. The tracked working tree remained unchanged during the audit. Temporary test code and raw evidence are under the ignored build/ and run/ directories; retain them if exact reproduction is needed.

Location relative to the repository	Contents
build/audit-20261024/	Capture/control code, Gradle init script, ordered request plans and logs, target-inspection agent, statistics, report source and figure crops.
run/audit-26.3/screenshots/	1,664 fresh PNG captures. Labels identify mode, scenario and frame index. Original pixel data is preserved.
run/audit-26.3/audit-*.json	Per-capture metadata and raw application frame-interval samples.
run/audit-26.3/qa-motion-*.json	Fresh GPU motion readbacks, including entity and world-border probes.
run/audit-26.3/qa-ssaa-area-*.json	Fresh synthetic GPU footprint-integration checks.
output/pdf/audit-evidence-manifest.json	Machine-readable source/native hashes, counts, timing summary, variability statistics and test results.

Build and run

Use Java 25, then run the project's Gradle wrapper build. The audit's temporary initialization script adds only the disposable harness and points runClient at the copied test world. The DLSS test uses the existing staged runtime via the dlssRuntimeDirectory property. Execute the saved JSON request plans through control.py in order; do not run multiple control plans concurrently.

Automated verification

98 automated tests across 23 suites completed with 0 failures/errors during this audit's build, including 16 shipped fragment-shader compilation cases. Fresh SSAA GPU checks passed all three cases. The scratch-harness startup errors were corrected before capture; they are not mod findings.

How images were prepared

All comparison panels use PNGs captured from the running Minecraft render target. Crops are specified in source-image pixel coordinates and enlarged with nearest-neighbor sampling. No generative editing, sharpening, denoising or color enhancement was applied to evidence panels. The PDF renderer may resample at fit-to-page scale; zoom to inspect the original stepped pixels.

The in-game client was shut down at the end. Production source files were not changed and no audit changes were pushed to GitHub.